Interview Question Bank - Allaudin Ansari

Campus Interview Question Bank: Allaudin Ansari (Fresher / Intern to Full-time)

Candidate background: B.Tech CSE (Honors in Quantum Computing), SRM IST, expected graduation May 2027. Projects: URL Shortener (backend), LexAI (RAG), AI-Powered Portfolio Optimization Engine (RL + MLOps), Real-Time E-Commerce Clickstream Lakehouse (data engineering), plus a Quantum Optimization internship (QUBO docking).

Purpose: Find out what the candidate is actually good at across software engineering, AI/ML, data engineering, and data science, so the candidate can be routed to the right role. This is not a single-role interview.

Basis for the question style: Typical Indian campus and off-campus fresher rounds (product companies and service companies). These usually include: (1) an online aptitude/coding round, (2) a technical interview on academics and projects, (3) DSA/coding problems, (4) a short system or ML/data design discussion, and (5) an HR/behavioral round. Questions below follow that flow. The priority order is: **depth on own projects > CS fundamentals > coding > behavioral**.

How to use this file: - Ask 6–8 questions per project, starting from the candidate's own explanation, then drill down. - Follow-up probes are marked with ➜. - "Good signal" notes say what a strong fresher answer looks like. A weak answer that recites the resume is itself a signal.

About these answers: Each answer is written as the candidate would give it, using only facts from the project notes and resumes. Where the project did not measure something, the answer says so instead of inventing a number. Items in [square brackets] are personal details the candidate must fill in from their own experience.

PART A: Project 1 — URL Shortener (Scalable Backend and Distributed System)

Stack claimed: Python, FastAPI, React, Docker, GitHub Actions, Redis, Celery, Prometheus/Grafana, PostgreSQL.

1. Walk me through what happens, step by step, when a user clicks a short link. ➜ *Where does the Redis lookup happen? What if Redis is down?*

   Answer: The browser sends `GET /{slug}` to the FastAPI service. The handler first checks Redis for the slug. On a hit, it returns a 307 redirect to the target URL straight away. On a miss, it queries PostgreSQL by slug, writes the result back to Redis with a 1-hour TTL, and then redirects. Click recording is not done in this request: the handler publishes a Celery task, so the user is not kept waiting for analytics. If Redis is down, the cache lookup fails and the app should fall through to PostgreSQL. The cost is higher latency and more database load, but redirects still work. I should be honest that I have not formally tested this failure mode with Redis stopped, so I would describe it as the intended design, not as a verified result.

2. Why did you choose a 307 redirect over a 301? ➜ *What breaks with 301 if you want click analytics?*

   Answer: A 301 is permanent, so browsers cache it. After the first visit, the browser goes straight to the destination and never calls my server again. That means I would lose click counts and could never change or deactivate a link for users who already visited. A 307 is a temporary redirect, so the browser asks the server every time. That keeps analytics accurate and lets link management work.

3. How is the short slug generated? Why base62? ➜ *What is the collision risk with random slugs vs. sequence-based encoding?*

   Answer: Each link gets a numeric id from a database sequence, and I encode that number in base62 (0-9, a-z, A-Z). Base62 is used because those characters are URL-safe and it produces short strings. Because the encoding is one-to-one, two different ids can never produce the same slug, so there is no collision by construction. With random slugs, you would need to check for collisions and retry, and the chance of collision grows as the table fills up.

4. Why did you use a database sequence instead of `SELECT MAX(id) + 1`? ➜ *Explain the race condition that would occur.*

   Answer: With `MAX(id)+1`, two requests can read the same maximum at the same moment and both try to insert the same next id. One of them fails or, without a unique constraint, you get duplicate data. A sequence (`nextval`) is handled atomically by the database, so every caller gets a different number even under concurrency. My test fires 50 concurrent creation requests and checks that all slugs are distinct.

5. What is cache-aside? Why did you choose it over write-through? ➜ *Where does your code populate the cache on link creation, and why?*

   Answer: In cache-aside, the application checks the cache first, and on a miss it reads the database and fills the cache. It fits a read-heavy workload where many links are never clicked, so I do not want to cache everything. One exception: when a link is created, I write it to the cache immediately, because a new link is usually shared and clicked within seconds. That is a write-through step on creation only.

6. What is negative caching? Why does the negative TTL (60 s) differ from the positive TTL (3600 s)?

   Answer: Negative caching stores the fact that a slug does not exist, so repeated requests for a bad slug do not hit PostgreSQL each time. The negative TTL is short (60 s) because a slug that was missing a moment ago might be created right after, and I do not want a false 404 cached for an hour. A real link is stable, so a 1-hour positive TTL is fine.

7. Why is click recording done through Celery and not inline in the redirect request?

   Answer: Writing a click row inside the redirect would make the redirect wait for the database write. If that write slowed down or failed, the redirect would be affected too. With Celery, the handler just publishes a message to Redis, and a separate worker writes to PostgreSQL. The redirect latency stays independent of analytics volume, which is the main point of that design.

8. What does “at-least-once delivery” mean? Could a click be counted twice? How would you make it idempotent?

   Answer: At-least-once means a message is delivered one or more times, and never silently lost. If a worker crashes after writing the row but before acknowledging, the task can run again, so yes, a click could be counted twice in rare cases. This is a trade-off I chose knowingly: double-counting one click is a minor analytics error. To make it idempotent, I would store a unique event id for each click and skip duplicates, or use a unique constraint on that id. I have not built that part yet, so I would describe it as a next step.

9. What does `acks_late` (or equivalent) guarantee in Celery? What happens if a worker crashes mid-task?

   Answer: With late acknowledgement, the message is removed from the queue only after the task finishes, not when a worker picks it up. If the worker dies mid-task, the message is still in the broker and another worker takes it. My task also has `max_retries=3` with a 5-second delay, so transient errors are retried and a permanently broken task does not retry forever.

10. Why did you apply rate limiting only to `POST /links` and not to the redirect endpoint?

    Answer: Creating a link writes to the database and uses the shared sequence, so abuse there costs real resources and can fill the table with junk. Redirects are the product itself. A popular link getting many clicks is the system working correctly, so rate-limiting redirects would punish success. The limit on creation is 10 requests per minute per IP, using slowapi.

11. Why is the rate-limit counter stored in Redis and not in process memory? ➜ *What happens with 3 API replicas?*

    Answer: With in-memory counters, each replica keeps its own count. A client could send 10 requests to each of 3 replicas and get 30 through, so the real limit would be multiplied. Storing the counter in Redis means all replicas share one count, so the limit holds regardless of how many instances are running.

12. You mention a foreign-key bug caught only by Postgres tests. Tell me that story. What did SQLite not enforce?

    Answer: My fast unit tests run on in-memory SQLite. In one version, a custom-alias click could be recorded with a `link_id` that did not exist in the links table. SQLite does not enforce foreign keys by default, so the unit tests passed. The separate Postgres test suite rejected the insert, which exposed the bug. The lesson for me is that a database constraint behaves differently across engines, so some tests have to run on the real database.

13. Why did you write both unit tests (SQLite) and a separate Postgres suite? What is the testing pyramid?

    Answer: The testing pyramid says to have many fast unit tests at the bottom, fewer integration tests in the middle, and a few end-to-end tests at the top. Unit tests run in seconds on SQLite, so I can run them constantly. The Postgres suite is smaller and slower, and it exists specifically to catch engine-specific behavior like foreign keys. The resume says the project has 90+ unit, integration, and end-to-end tests in total.

14. Explain your “50 concurrent requests” test. Did you use real concurrency (`asyncio.gather`) or sequential calls? Why does that matter?

    Answer: The test launches the creation requests together with `asyncio.gather`, over separate database connections, so they really overlap. Sending them one after another would never trigger a race condition, so the result would prove nothing about concurrency. The test asserts that all 50 slugs come back distinct.

15. What is cardinality in Prometheus? Why must the `path` label use the route template, not the raw slug?

    Answer: Cardinality is the number of distinct label combinations a metric has, and Prometheus stores one time series per combination. If I labeled by the raw slug, every link would create a new permanent series, which can grow until the monitoring system runs out of memory. I label by the route template, such as `/links/{slug}`, which gives a fixed number of series.

16. Explain `rate(http_requests_total[1m])`. Why does `rate()` require a counter?

    Answer: `http_requests_total` is a counter that only goes up. `rate(...[1m])` takes the increase over the last minute and divides by the time, which gives requests per second. It needs a counter because a counter's increase over time is meaningful. A gauge can go up and down, so its rate is not meaningful.

17. What is the difference between a liveness and a readiness probe? Does your `/health` do one or both?

    Answer: Liveness asks whether the process is alive, and failing it usually means restart. Readiness asks whether the service can take traffic right now, and failing it means stop routing to it. My `/health` endpoint is a simple liveness check. I did not implement a separate readiness check that verifies the database and Redis connections, so I would add that before any real deployment.

18. Explain the request ID middleware. Why use `contextvars` instead of passing the ID through every function?

    Answer: The middleware reads `X-Request-ID` from the request, or generates a UUID if it is missing. It stores that id in a `contextvars` variable, so any log line during that request can include it without passing it through every function call. `contextvars` keeps values separate per request even when many requests run concurrently on the same event loop. The id is also echoed in the response so one request can be traced across services.

19. How does input validation work? Why reject `"health"` as a custom slug?

    Answer: Pydantic checks the request shape and types, and a custom validator checks the slug characters (letters, digits, hyphen, underscore). I also reject reserved words. `"health"` is a valid string, but `/health` is a real route, so a link with that slug would clash with the endpoint and break it. The reserved list covers words like `links`, `health`, `metrics`, and `docs`.

20. Why must custom aliases be validated against a reserved list? What could go wrong otherwise?

    Answer: The shortener's own routes and the redirect path share the same URL space. If a user takes `/metrics` as a slug, the redirect route might be shadowed or the metrics endpoint might become unreachable. Validation before the database write keeps the namespace safe.

21. How would you prevent a user from shortening a URL that points back to your own shortener (redirect loop)?

    Answer: The creation handler parses the target URL and compares its host with the shortener's own host. If they match, it returns a 400 with a clear message. This blocks direct loops. It does not catch a chain through a third-party site, so I would also consider limiting redirect chains or checking targets in a background job.

22. Your claim is "horizontally scalable." What specifically makes it scalable, and what is still a single point of failure? ➜ *Single Redis? Single Postgres?*

    Answer: The API is stateless: session state is not kept in process memory, and the rate-limit counter and cache live in Redis. So I can run several API replicas behind a load balancer. However, my current setup has one Redis and one PostgreSQL instance, so both are single points of failure. Scaling the database would need read replicas or partitioning, and Redis would need replication or clustering. I should be careful to say "the API tier is horizontally scalable," not the whole system.

23. How would you measure p95 latency of redirects? What would you change if p95 were 300 ms?

    Answer: I would record a histogram of redirect duration in Prometheus, using buckets such as 10, 25, 50, 100, 200, 500 ms, and compute p95 with `histogram_quantile(0.95, ...)` over a rate window. I have not measured p95 under load, so I cannot give a real number. If it were 300 ms, I would first check whether it was cache misses (database time), then whether the database query was slow, then whether the event loop was blocked.

24. Walk me through your GitHub Actions pipeline. What runs on every PR and what runs only on merge?

    Answer: On every push and pull request, the CI workflow runs linting and the test suite. A merge to main triggers the deployment-related steps, which build the Docker image. I describe the exact job split from my workflow file, and I would double-check it before the interview, since the file is the source of truth.

25. What is Docker multi-stage build, and why is it useful for a Python service?

Answer: A multi-stage build uses one stage to install dependencies and build things, then copies only the finished packages into a smaller runtime image. The final image does not include compilers or build tools. That makes the image smaller, faster to deploy, and less exposed if the container is compromised.

26. What would you change first if traffic grew 100x? Justify the order.

    Answer: First I would look at the cache: a higher cache hit rate reduces database load the most. Second, I would add read replicas for PostgreSQL, since redirect misses and stats queries are reads. Third, I would run more API replicas behind a load balancer. I would check each change against metrics before the next, instead of changing everything at once.

PART B: Project 2 — LexAI (Legal RAG System)

Stack claimed: Python, LangChain (LCEL), FastAPI, React.js, FAISS, BM25, Ollama, CrossEncoder, SSE.

1. Explain RAG in simple words. Why is it better than asking an LLM directly for legal questions?

   Answer: RAG stands for Retrieval-Augmented Generation. Before the language model answers, the system searches a trusted set of documents and passes the relevant passages to the model, which is told to answer only from them. For legal questions, this matters because a plain LLM can give a plausible but wrong section number from memory, and it cannot show a source. With RAG, every answer can cite the document and page it came from.

2. Walk me through the ingestion pipeline: PDF → chunks → embeddings → index. Where can it fail?

   Answer: The pipeline loads each PDF page with `pypdf` and skips near-empty pages. It splits the text into semantic chunks, embeds each chunk with the embedding model, and builds two indexes: a FAISS vector index and a BM25 keyword index. Failures can happen in PDF parsing (corrupt file or scanned page without text), in the embedding call (rate limits or the model not running), or in the index write. My code handles these cases: a bad page is skipped, embedding quota errors trigger a wait and retry, and the BM25 index can still be built if embeddings fail. A backup of the old index is kept before overwriting.

3. What is semantic chunking? How is it different from fixed-size chunking? ➜ *What is the percentile breakpoint doing?*

   Answer: Fixed-size chunking cuts text every N characters, even in the middle of a clause. Semantic chunking first splits into sentences, embeds a small window around each sentence, and measures how far apart consecutive window embeddings are. Where the distance is large, the topic has shifted, so I place a chunk boundary. The breakpoint is the 90th percentile of those distances, which means the document is split at its most significant topic shifts. Using a percentile rather than a fixed number lets the threshold adapt to each document. Chunks shorter than 400 characters are merged, and chunks longer than 1800 characters are force-split.

4. Why do you use a 220-character overlap in hard splits? What is lost without it?

   Answer: When a chunk is force-split because it is too long, the boundary can cut a sentence in half. The 220-character overlap means the end of one chunk repeats at the start of the next, so a clause that straddles the boundary is still complete in at least one chunk. Without overlap, a sentence like “the accused shall be liable” could be separated from the words that say what it is liable for.

5. What is an embedding? What does cosine similarity measure, and why normalize vectors before FAISS?

   Answer: An embedding is a list of numbers that represents the meaning of a text, so similar meanings end up close together. Cosine similarity measures the angle between two vectors, ignoring their length. I normalise every vector to length 1 before storing and searching. After normalisation, the inner product equals cosine similarity, so I can use FAISS’s fast inner-product index and still get cosine-based ranking. Without normalising, longer vectors would score higher for reasons unrelated to meaning.

6. Why `IndexFlatIP` and not an approximate index like HNSW or IVF? At what corpus size would you switch?

   Answer: `IndexFlatIP` is exact: it compares the query with every stored vector. For a legal corpus of a few thousand to tens of thousands of chunks, that is fast enough, and the results are guaranteed correct. An approximate index trades a little accuracy for speed on very large collections. I would consider switching when query latency becomes a problem with my real corpus size. I have not measured that threshold, so I would not give a specific number.

7. What is BM25? Why is it still useful when you have dense embeddings? ➜ *Give an example where BM25 wins (e.g., “Section 302”).*

   Answer: BM25 is a keyword-ranking method. It scores documents higher when they contain the query’s rare terms, with diminishing returns for repeated words. It is useful because embeddings can treat “Section 302” and “Section 304” as very similar, since the text looks alike, but in law they are different offences. BM25 matches the exact token

"302", so it finds the right section. Dense search is better for paraphrases, such as "penalty for stealing" matching "punishment for theft." Together they cover both cases.

8. Why filter stopwords for BM25 in this corpus? What went wrong without it?

   Answer: Legal text repeats words like "shall", "under", and "the" in almost every chunk. BM25's usual weighting handles common words on large general corpora, but on a small, repetitive legal corpus those words kept rising in rankings. I removed stopwords and checked the result: without that step, an unrelated IT Act chunk outranked the murder-punishment chunk for the query "what is the punishment for murder." That is why stopword removal is in the code.

9. What is Reciprocal Rank Fusion? Write the formula. Why use rank instead of raw scores?

   Answer: RRF combines ranked lists by summing 1/(k + rank) for each document across the lists it appears in. I use k = 50 from the settings. Raw scores are not comparable: BM25 scores are unbounded and corpus-dependent, while cosine similarity is between -1 and 1. Ranks are on the same scale for both, so no normalisation is needed. A document ranked well by both methods accumulates two contributions, which is what I want.

10. What is HyDE? Why would generating a hypothetical answer improve retrieval? What if the LLM hallucinates?

    Answer: HyDE asks the LLM to write a short hypothetical answer in the style of a statute or judgment, then embeds that answer for retrieval. A question and its answer are phrased differently, so the answer's embedding often lands closer to real statutory text. If the LLM hallucinates, the damage is limited: I search with the original question as well, and the fused ranking combines both. If HyDE fails completely, the request continues without it.

11. What is the difference between a bi-encoder and a cross-encoder? Why rerank only the top ~20?

    Answer: A bi-encoder embeds the query and each document separately, so document vectors can be precomputed and searched fast. A cross-encoder reads the query and one document together, so it is more accurate, but it has to run again for every pair. Running it on the whole corpus would be far too slow for each question, so I run it only on the roughly 20 candidates that retrieval returns.

12. Why does your pipeline retrieve broad and rerank narrow? Explain the latency vs. accuracy trade-off.

    Answer: The cheap stage narrows thousands of chunks to about 20 candidates, then the expensive, accurate stage picks the best 5 to send to the LLM. The trade-off is that the first stage can miss a good chunk, so the candidate pool has to be large enough to include it. The CrossEncoder is the bge-reranker-v2-m3 model. Its cost is why it is loaded once at startup rather than per request.

13. How do you prevent the LLM from making up section numbers? Show me the grounding prompt.

    Answer: The system prompt tells the model to answer using only the retrieved context, to say plainly when the context is not enough, to cite sources in a format like "(IPC 1860.pdf, p.45)", and not to invent section numbers or case names. Each chunk is passed with a numbered header that includes the file name, page, and law type, so citation is easy for the model. Prompting reduces hallucination but does not remove it, so I rely on the citations to verify answers. I can show the exact prompt text from the code.

14. What is "graceful degradation"? Which parts of your pipeline fail softly and which fail hard, and why?

    Answer: Graceful degradation means that when an enhancement fails, the system still answers at lower quality instead of failing. HyDE and reranking fail softly: if HyDE fails, I retrieve with the question alone, and if reranking fails, I use the RRF order. Retrieval itself fails hard, with an explicit error, because an answer without any retrieved evidence would be ungrounded, and I do not want to present that as a sourced answer.

15. How do you handle conversation history? Why cap it at the last 8 messages?

    Answer: Each session keeps its own in-memory message history. The last 8 messages, which is four user-assistant pairs, are added to each new prompt so follow-up questions make sense. I cap it because the context window is limited, and old turns would take space from the retrieved legal text, which matters more for answer quality. Sessions expire after a TTL. The store is in-process, so it does not survive a restart, which I list as a limitation.

16. What is Server-Sent Events? Why not WebSockets for this use case?

    Answer: SSE is a one-way stream from server to client over a normal HTTP response, with messages formatted as `event:` and `data:` lines separated by a blank line. WebSockets are two-way and need a separate protocol. My interaction is one question followed by a stream of answer tokens, so one-way streaming is enough. SSE is simpler and works over plain HTTP.

17. Why did you use `POST` for the SSE endpoint instead of the browser's `EventSource`?

Answer: `EventSource` can only make GET requests with no body. My endpoint needs a JSON body with the question and session id. So I use `fetch` with a POST and read the response body as a stream, and I parse the SSE frames myself.

18. Explain how your frontend handles a single SSE frame split across two network reads.

    Answer: A network read can end anywhere, so one frame may arrive in two pieces. I keep a buffer of text, split it on the blank-line separator, and hold back the last incomplete piece until the next read completes it. Only complete frames are parsed. I also drop a frame whose JSON is malformed instead of crashing the whole stream. I wrote a test that splits one frame across several simulated reads to check this.

19. What is the point of the retrieval benchmark? Explain Precision@5, Recall@5, and nDCG@10 with a simple example.

    Answer: The benchmark checks whether the retrieval design actually helps, instead of assuming it does. Using a simple example: if the top 5 results contain 2 relevant chunks out of 5 retrieved, precision@5 is 0.4. If there are 10 relevant spans in total and the system found 2 of them in those 5 results, recall@5 is 0.2. nDCG@10 also rewards relevant results near the top, so a relevant result at rank 1 counts more than the same result at rank 9. My benchmark measures these at the character-span level, following the LegalBench-RAG format.

20. Your benchmark shows hybrid retrieval beats FAISS alone by a large margin. Why do you think that is? ➜ *What would you test to be sure it is not an artifact of the chunker?*

    Answer: My benchmark numbers: on 1,171 queries across privacy_qa and contractnli, baseline FAISS gets recall@5 of about 0.003, hybrid about 0.11, and hybrid plus reranker about 0.19. The baseline is very weak, which is partly because the benchmark's dense-only setup is simple and the queries are hard. To check that the chunker is not responsible, I would run the same three configurations with two or three different chunkers and see whether the ranking between them stays the same. I would also check that the two benchmark runs agree, which they roughly do.

21. Why is the benchmark's chunker deliberately simple? What confound does that avoid?

    Answer: The benchmark uses fixed-size chunks with tracked character offsets, not the production semantic chunker. If each configuration used a different chunker, a difference in scores could come from chunking, not from retrieval. Using one chunker for all three configurations isolates the retrieval method, which is the variable I want to measure. The cost is that the benchmark does not test the exact production chunking.

22. Your RAGAS-style evaluation uses token-overlap F1, not the real RAGAS library. Why? What are the limitations of token overlap?

    Answer: The current evaluation script computes faithfulness and relevancy as token-overlap F1 scores against reference answers. I chose this because it runs offline and does not spend LLM quota. Its limitation is that it measures word overlap, so a correct paraphrase can score low, and a wrong answer that copies words from the context can score high. The real RAGAS metrics use an LLM as judge to catch that. I also have a compatibility fix for a known upstream dependency bug in the RAGAS library, so the step to switch is available, but it is not what runs today.

23. What does a "faithfulness" score measure? How would you check it with an LLM judge?

    Answer: Faithfulness measures whether each claim in the answer is supported by the retrieved context. With an LLM judge, I would split the answer into claims, ask the judge for each claim whether the context supports it, and report the fraction supported. I would also sample some judgments by hand to check the judge itself.

24. The configuration suggests Google embeddings, but the code only builds Ollama clients. Why would that be? (Tests whether the candidate actually knows their code.)

    Answer: This is a real gap in my code. The settings and docstrings mention Google as a provider, but in the current code the provider functions build only the local Ollama clients. So the project runs fully locally, but the Google path is not wired up. I would either implement that branch or remove it from the configuration, so the docs match the code. I should not claim it works.

25. What is the difference between `task_type="retrieval_query"` and `"retrieval_document"`? What happens if they are swapped?

    Answer: Some embedding models produce different representations for a search query and for a document to be searched. I use `retrieval_document` when indexing chunks and `retrieval_query` when embedding the user question. If they are swapped, the code still runs, and retrieval quality drops silently. That is why I documented it in the retriever.

26. Why did you write your own FAISS wrapper instead of using `langchain_community`? What is the dependency risk?

    Answer: The `langchain_community` package is no longer maintained, and building a core component on an unmaintained package is a risk. My wrapper is small. It uses `faiss` directly, keeps a list of documents in the same order as the index, and saves both to disk. The cost is a few dozen lines of code I need to test myself. I also wrote a semantic chunker for the same reason, since the experimental package it came from is unmaintained.

27. How would you add a new law PDF to the index without re-embedding everything?

    Answer: The current pipeline rebuilds the index from all PDFs, using a checkpoint to skip completed files, which helps after an interruption. For a new law, the right approach is to embed only the new file's chunks and add them to the existing FAISS index and BM25 index, then save both. The index structure supports adding documents, but I would need to verify that the checkpoint and the append path handle the document ids correctly. I have not built that incremental path yet.

28. What is idempotent ingestion? How does your checkpoint and backup-before-overwrite work?

    Answer: Idempotent ingestion means that running it again gives the same final result and does not corrupt data. Checkpointing records which files finished, so a restart skips them. Before writing a new index, the old one is renamed to a timestamped backup, so a bad run can be recovered. The pipeline also refuses to overwrite anything if it produced zero chunks.

29. How would you deploy this system with a rate limit on the LLM API?

    Answer: I would put the LLM and embedding calls behind a queue or a semaphore that limits concurrent requests, use exponential backoff on 429 errors, and cache answers for repeated questions. I would also set per-user rate limits at the API layer, the same way I did in the URL shortener. Deployment would use Docker, with the model endpoint and index files configured through environment settings.

30. What would you improve first to raise answer quality: chunking, retrieval, reranking, or the prompt? Justify.

    Answer: I would start with retrieval measurement. The benchmark shows recall is the weakest number, so I would look at which relevant chunks are missed and why, before changing anything. If the right chunk is never retrieved, no prompt change can help. Chunking would be the next thing to test, using the benchmark to compare chunkers, because chunk boundaries set what can be retrieved at all.

PART C: Project 3 — AI-Powered Portfolio Optimization Engine

Stack claimed: Python, PyTorch, FastAPI, MLflow, Docker, Airflow, PPO, SAC, Markowitz, Black-Litterman, Stable-Baselines3.

1. Explain the project in 60 seconds, starting from the problem, not the tools.

   Answer: The problem is deciding how to split money across a set of stocks over time. I built a system that uses reinforcement learning agents, PPO and SAC, to set portfolio weights for 40 Nifty50 and S&P500 stocks. I compare them against classical baselines, Markowitz and Black-Litterman, and an equal-weight benchmark. I evaluate with walk-forward backtesting, measure risk-adjusted returns with Sharpe and Sortino, and wrap the training and serving in an MLOps pipeline with MLflow, Airflow, and FastAPI. The agent is trained on past data and tested on later windows it has not seen.

2. What is a Sharpe ratio? Why annualise with √252?

   Answer: Sharpe is the mean excess return divided by the standard deviation of returns. It measures return per unit of total risk. Daily Sharpe is annualised by multiplying by √252, because returns are roughly independent day to day and their standard deviation grows with the square root of time, while the mean grows linearly. There are 252 trading days in a year.

3. What is the difference between Sharpe and Sortino? Why use Sortino as an evaluation metric?

   Answer: Sharpe penalises all volatility, including large positive days. Sortino uses only downside deviation, so big gains are not counted as risk. I use Sortino for episode-level evaluation and in the backtest report, alongside Sharpe. I do not use it as the per-step training reward, because a per-step Sortino number is too noisy. The per-step reward is log return minus a transaction cost.

4. What is maximum drawdown? Compute it for an equity curve 100 → 150 → 90 → 200.

   Answer: Maximum drawdown is the largest drop from a running peak to a later trough. Here the peak before the fall is 150 and the trough is 90. The drawdown is (90 − 150) / 150 = −40%. The later recovery to 200 does not change the maximum drawdown, which is −40%.

5. What is CVaR, and why is it preferred over VaR by risk managers?

   Answer: VaR at 95% is the loss threshold that 95% of days stay below. CVaR is the average loss on the worst 5% of days, so it describes how bad the tail is, not only where it starts. Two portfolios can have the same VaR but very different tails. CVaR is considered a more coherent risk measure, and it cannot be gamed by hiding losses just beyond

the VaR cutoff. In my results, the RL agent's CVaR at 95% was around −1.8% to −2.7%, compared with −3.2% to −3.8% for the benchmarks, depending on the run.

6. Why are log returns used instead of simple returns? Show that two daily log returns sum to a 2-day log return.

   Answer: Log returns add over time, and they are roughly symmetric. ln(P2/P0) = ln(P1/P0) + ln(P2/P1), so two daily log returns sum exactly to the two-day log return. Simple returns compound multiplicatively, so they do not add. Log returns also let the code build the equity curve as exp of a cumulative sum.

7. What is look-ahead bias? Name three places your project prevents it.

   Answer: Look-ahead bias means using information that would not have been available at decision time. My project prevents it in three places. First, the feature scaler is fitted only on the training window, not on the full dataset. Second, the Markowitz covariance is estimated only from a backward-looking window before each rebalance date. Third, the walk-forward backtest keeps test periods strictly after training periods, and the RL environment only starts episodes inside the training period.

8. Why did you use `RobustScaler` instead of `StandardScaler`? Where is the scaler fitted?

   Answer: Financial returns have fat tails, so a crash day can distort the mean and standard deviation that `StandardScaler` uses. `RobustScaler` uses the median and interquartile range, which are much less affected by outliers. The scaler is fitted only on the training slice, and the validation and test slices only call transform.

9. What is walk-forward backtesting? Why is a single train-test split insufficient for financial data?

   Answer: Walk-forward backtesting trains on a window, tests on the next window, then slides forward and repeats. My windows are 504 training days, 126 test days, and a step of 21 days. A single split tests only one market regime. A strategy can look good in a bull market and fail in a crisis, and one split would never show that. Walk-forward gives a result for many windows.

10. Why is your Sharpe of 1.4 vs. 0.6 a claim that needs scrutiny? What would you check? ➜ *Survivorship bias, transaction costs, data snooping, out-of-sample test.*

    Answer: I agree it needs scrutiny. The 1.4 figure is for the RL agent's walk-forward test windows, compared with about 0.6 for the Nifty50 benchmark and 0.74 to 0.77 for Markowitz, depending on the run. The main questions are: survivorship bias, since the stock universe is today's index members; transaction costs, which I model as 0.1% of turnover, which might be low for real execution; data snooping, since I tried several configurations and reported the best; and whether the test windows were used for model choice. I would check survivorship with point-in-time index membership, test higher costs, and keep one final holdout period that is never used for decisions.

11. What is a p-value? What does $p < 0.05$ mean here, and what does it NOT mean?

    Answer: A p-value is the probability of seeing a result at least this extreme if the null hypothesis were true. I ran a two-sided t-test on monthly excess returns of the RL agent over equal-weight. $p < 0.05$ means the outperformance would be unlikely if there were no real edge. It does not mean there is a 95% chance the strategy works, and it does not mean the strategy will keep working. I should also be open that my two documents report different p-values, 0.024 in one and a much smaller number in the other, from different runs. I would report the exact run I am quoting.

12. What is Markowitz mean-variance optimization? What are its two main inputs?

    Answer: Markowitz finds weights that give the highest expected return for a given level of risk, or the lowest risk for a given return. The two inputs are the expected returns for each asset and the covariance matrix, which captures how assets move together. The diversification effect comes from the covariance: assets that do not move together reduce total risk.

13. Why is the covariance matrix noisy with 40 assets and 504 days? What does Ledoit-Wolf shrinkage do?

    Answer: With 40 assets, the covariance matrix has 820 unique entries, estimated from about 504 observations. That makes the estimate noisy and sometimes nearly singular, which gives extreme and unstable weights. Ledoit-Wolf shrinks the sample covariance toward a simple, stable target, with the amount of shrinkage estimated analytically. It keeps most of the real signal and reduces noise. I use scikit-learn's `LedoitWolf` class.

14. Why cap maximum weight at 40% per asset?

    Answer: Without a cap, the optimiser can concentrate almost all capital in one or two assets that looked great in the sample. Those solutions are overfit and fragile. The 40% cap forces some diversification even when the raw numbers suggest otherwise. It is a design choice, and I can tell the interviewer that a different cap would change the results.

15. What is reinforcement learning? Define agent, environment, state, action, and reward for this project.

Answer: Reinforcement learning trains an agent by trial and error, using rewards from an environment. Here the agent is the PPO or SAC policy. The environment is my `PortfolioEnv`, which replays historical prices. The state is recent features for each asset, such as momentum, volatility, and RSI, plus the current weights. The action is a vector of 40 raw numbers that are turned into weights through softmax. The reward is the day's log return minus a transaction cost based on turnover.

16. Explain PPO in simple terms. What does "clipping" prevent?

    Answer: PPO is a policy-gradient method that improves the policy using recent experience. The problem it solves is that a single large update can make the policy much worse. PPO clips the ratio between the new and old policy's probabilities, so an update cannot move the policy too far in one step. My config uses a clip range of 0.2, and training ran for 100,000 steps.

17. What is the difference between on-policy (PPO) and off-policy (SAC) learning? Why would you choose one?

    Answer: On-policy methods like PPO learn only from experience collected by the current policy, and that data is thrown away after updates. Off-policy methods like SAC learn from a replay buffer of past experience, so they reuse data and are often more sample-efficient. SAC also adds an entropy bonus that keeps the policy exploring. I use PPO as the primary algorithm because it is stable and simple to tune, and SAC as a challenger, trained for 150,000 steps.

18. Why is the action output passed through softmax? Why not let the network output weights directly?

    Answer: A portfolio needs weights that are non-negative and sum to 1. If the network output weights directly, it has no reason to produce valid weights. Softmax turns any real-valued vector into positive numbers that sum to 1, so every action is a valid portfolio and training stays smooth. I use the standard numerically stable form by subtracting the maximum before exponentiating.

19. Why does the reward include a transaction cost? What behavior would the agent learn without it?

    Answer: Without a cost, the agent has no reason to trade less. It could reallocate every day to chase tiny signals and look profitable in a frictionless simulation. Including 0.1% of turnover in the reward teaches it that trades must justify their cost. That is the same trade-off a real trader faces.

20. Why is the episode start date randomised? What would overfitting look like?

    Answer: If every episode started at the same date, the agent could memorise one path of prices. Random starts within the training period expose it to many market situations, so it learns a more general strategy. Overfitting would look like a very high training reward with much weaker performance on validation windows. Random starts are restricted to the training period, so validation and test data are never used in training.

21. What is reward hacking? Give an example in this trading setting.

    Answer: Reward hacking is when the agent finds a way to get a high reward that does not match what you wanted. In trading, if the reward were only the return, the agent might take huge leveraged bets that look good on average and fail badly in a crash. My reward includes a cost and is based on log returns, and I track drawdown and CVaR separately, so a single number is not the only check.

22. What is an MLOps pipeline? Explain what MLflow tracks: experiments, runs, params, metrics, artifacts, and the model registry.

    Answer: MLOps is the practice of tracking, deploying, and monitoring models reliably. MLflow organises work into experiments, which group related training attempts. Each run records its parameters, metrics such as validation Sharpe, and artifacts like the trained model file. The model registry holds named model versions, and an alias such as `champion` points to the version in production. The serving code loads the model by alias, so promotion does not require code changes.

23. What is champion-challenger promotion? Why a 5% threshold rather than "any improvement"?

    Answer: The current production model is the champion. A newly trained model is a challenger, and it replaces the champion only if its validation Sharpe is at least 5% higher. Retraining has randomness, so small differences are often noise. A 5% margin stops the system from swapping models back and forth on noise. The first model is promoted automatically because there is no champion yet.

24. How does your system detect model drift? Why is the threshold 0.5?

    Answer: The API keeps a rolling window of the last 30 daily portfolio returns and computes a rolling 30-day Sharpe. That value is exported as a Prometheus gauge. If it drops below 0.5, a Grafana alert fires, which triggers the retraining DAG. The threshold is below the backtested Sharpe of about 1.4 to 1.5, so ordinary noise does not trigger it constantly. I should be honest that this loop is built and wired, but I have not observed it run on real live traffic over a long period.

25. Why do Airflow tasks only make HTTP calls to the API instead of running training directly? What are the benefits?

    Answer: Airflow's job is scheduling and sequencing. If it also ran PyTorch and Stable-Baselines3, its image would become very large and its dependencies could conflict with Airflow's own. Its tasks are simple HTTP calls to `/retrain/*` endpoints on the API, which has the ML libraries installed. The Airflow image stays small, and the training code can change without touching the scheduler.

26. Why is the training endpoint guarded by a lock and a 409 response instead of async?

    Answer: Training is CPU-heavy. Running it inside an `async` handler would block the event loop, so `/health` and `/predict` would stop responding during training. Instead, a lock-protected flag tracks whether training is running, and a second request gets a 409 Conflict immediately instead of starting a second heavy job. The endpoint still blocks while training runs, so it is a deliberate trade-off.

27. What is the difference between `async def` and running CPU-heavy work? Why does it block the event loop?

    Answer: `async def` lets a single thread pause while it waits for I/O, such as reading a file or a network response, and serve other requests meanwhile. CPU-heavy work does not wait, so it keeps the one thread busy and nothing else gets served until it finishes. For CPU-heavy work, the right options are a separate thread, a process pool, or a background worker.

28. What is dropout? Why is it used in the LSTM forecaster? Did you actually use an LSTM in the final pipeline?

    Answer: Dropout randomly zeroes some neurons during training, so the network does not rely on any single path, which helps generalisation on noisy financial data. The LSTM forecaster is a separate model: it predicts next-day returns from 30 days of features and is compared against an ARIMA baseline. The final decision-making component in the portfolio system is the RL agent, not the LSTM. I should not claim the LSTM drives the final allocation.

29. What is early stopping? What is the patience parameter?

    Answer: Early stopping monitors validation loss each epoch, saves the best weights, and stops when the loss has not improved for a set number of epochs. That number is the patience. My forecaster uses a patience of 7, and then it reloads the best saved weights before testing. It prevents overfitting without guessing the right number of epochs in advance.

30. Would you trust this strategy with real money? What evidence would be required first?

    Answer: No, not yet. Before real money, I would need a final holdout period never used for any decision, realistic costs including slippage and taxes, point-in-time data to remove survivorship bias, a check that the results hold across different cost assumptions and universes, and a paper-trading period with live data. Right now the results come from historical backtests on a project, which is a useful research result but not proof of a tradable strategy.

PART D: Project 4 — Real-Time E-Commerce Clickstream Lakehouse

Stack claimed: Python, Apache Kafka, Spark Structured Streaming, Delta Lake, Airflow, PostgreSQL, dbt, Metabase, Docker.

1. Describe the architecture from producer to dashboard. Why is each tool there?

    Answer: A Python producer generates clickstream events and sends them to a Kafka topic, `ecommerce.clickstream`, with 3 partitions. Spark Structured Streaming reads the topic, validates and parses the events, and writes them to Delta Lake Bronze. A Silver job deduplicates and enriches the data. A Gold job computes windowed aggregates. Airflow schedules the Silver and Gold steps, then syncs the results to PostgreSQL. dbt builds a star schema in Postgres, and Metabase shows dashboards. Kafka decouples the producer from processing, Spark does the streaming work, Delta gives reliable lake tables, and Postgres serves dashboards.

2. What is the medallion architecture? What is the purpose of Bronze, and why keep it "dumb"?

    Answer: Medallion means layering data into Bronze (raw), Silver (cleaned and deduplicated), and Gold (business aggregates). Bronze stores events as they came in, with only the parsing and validation needed to structure them. Keeping it simple means I can replay the whole pipeline from Bronze if a bug is found later. Transformation logic goes in Silver and Gold, where mistakes are cheaper to fix.

3. What is Kafka? Explain topic, partition, producer, consumer, and offset.

    Answer: Kafka is a distributed log for event streams. A topic is a named stream. A topic is split into partitions, each an ordered, append-only log. Producers write messages to a topic, and consumers read them. Each message in a partition has an offset, its position in that partition. Consumers track their offsets, so they can resume after a restart.

4. Why do you key Kafka messages by `user_id`? What ordering guarantee does it give, and what does it NOT give?

   Answer: The key determines the partition, so all events for one user go to the same partition, in the order they were produced. That is what makes per-user sequences and session logic workable downstream. It does not give ordering across different users, and it does not give ordering across partitions for the whole topic. Ordering is only guaranteed within one partition.

5. What is a consumer group? How does parallelism relate to partition count?

   Answer: A consumer group is a set of consumers that share a topic's partitions, so each partition is read by only one consumer in the group at a time. Parallelism is capped by the number of partitions. My topic has 3 partitions, so at most three readers in one group can work in parallel. Spark's Kafka connector manages this internally.

6. What is the difference between `acks=0`, `acks=1`, and `acks=all`?

   Answer: `acks=0` means the producer does not wait for any acknowledgement, which is fastest but can lose data. `acks=1` waits for the partition leader to write. `acks=all` waits for all in-sync replicas, which is the strongest guarantee. My producer uses `acks=all` with retries. My topic has a replication factor of 1, though, so there is no real replica to protect against a broker failure. I should say that clearly.

7. What is Spark? Explain driver, executor, and the difference between a transformation and an action.

   Answer: Spark is a distributed engine for large-scale data processing. The driver runs my application and plans the work. Executors run the tasks in parallel on the data partitions. A transformation, such as `select`, `filter`, or `groupBy`, describes new data and is lazy. An action, such as `count`, `collect`, or starting a streaming query, triggers execution.

8. What is lazy evaluation? Why does it matter for Spark performance?

   Answer: Spark records transformations as a plan and only runs them when an action needs the result. That lets Spark optimise the whole plan, for example by pruning columns and combining steps, before reading any data. It also means a mistake in a transformation may show up only when the action runs.

9. What is Structured Streaming? Explain append, update, and complete output modes. Why does Gold use `update`?

   Answer: Structured Streaming treats a stream as a table that keeps growing, and uses the DataFrame API. In append mode, only new rows are written, which is right for events that never change. Update mode writes only rows that changed since the last trigger. Complete mode rewrites the whole result. Gold uses update because a window's count for the last hour keeps changing as late events arrive, so the existing row must be revised, not duplicated.

10. What is a watermark? Explain the trade-off between correctness and latency with your 10-minute value.

    Answer: A watermark tells Spark how late an event can arrive and still be counted. With a 10-minute watermark, Spark waits up to 10 minutes for late events before finalising a window and discarding its state. A longer watermark captures more late data, which is more correct, but holds state longer and delays results. A shorter one gives faster results but drops more late events. I chose 10 minutes because the producer simulates late events up to 10 minutes late, so the value matches the test data.

11. What is `dropDuplicatesWithinWatermark`? Why does it fail on a static DataFrame?

    Answer: It removes duplicate rows by key and keeps only the keys seen within the watermark window, which bounds memory. It needs an active watermark, which only exists in a streaming query. On a static DataFrame there is no watermark, so Spark rejects the call. I deduplicate by event id and timestamp, and I verified it end to end by injecting duplicates and checking that they did not reach Silver.

12. What is a checkpoint? What happens if the checkpoint directory is lost?

    Answer: A checkpoint stores the Kafka offsets already processed and any streaming state, so a restarted query resumes where it stopped. Each query has its own checkpoint directory. If the checkpoint is lost, the job starts from the earliest offset again, which means reprocessing and duplicate data downstream. That is a real risk, so the checkpoint location has to be on persistent storage.

13. What is the difference between `processingTime` and `availableNow` triggers? Why did you switch?

    Answer: `processingTime` runs a micro-batch every N seconds, forever. `availableNow` processes everything currently available and then stops. I started with continuous streaming, but the Silver and Gold steps did not need to run all the time. Switching them to `availableNow` lets Airflow schedule them every two minutes, which uses resources only when there is work, and it is easier to monitor.

14. Why partition Silver by `event_date`? What is partition pruning?

Answer: Most downstream queries filter or group by date. Partitioning by date stores each day in its own folder, so a query for one day reads only that partition. Partition pruning is the engine skipping the other partitions. It is a physical layout choice based on query patterns.

15. What is data skew? Give an example in this dataset. How would you detect it?

    Answer: Data skew is when a few keys or partitions hold much more data than the rest, so one task becomes the bottleneck. In this dataset, a very active user or a single day with a traffic spike could skew one partition. I would detect it by checking the event count per partition or per key, and in the Spark UI, by looking for tasks that take much longer than the others.

16. What is shuffle? Why is `spark.sql.shuffle.partitions` set to 4 here?

    Answer: A shuffle moves data across executors so that rows with the same key end up together, which happens during `groupBy` and joins. The shuffle partition count sets how many partitions the result is split into. My setup runs a single worker with two cores, so 4 is a reasonable small number. The default of 200 would create many tiny tasks for little data. I chose it for this local cluster, not as a production value.

17. What is Delta Lake? How is it different from plain Parquet?

    Answer: Delta Lake stores data as Parquet files plus a transaction log. The log records which files belong to the table, so writes are atomic and readers never see half-written data. It also gives schema enforcement, time travel, and the ability to use the same table for streaming and batch. Plain Parquet has none of these transaction guarantees.

18. What is the `_delta_log`? How did your Airflow freshness check use it without starting Spark?

    Answer: The `_delta_log` folder holds JSON commit files, one per transaction, that describe the table's state. The freshness check finds the newest commit file and compares its modification time with the current time. If it is too old, the DAG fails. That check uses plain file metadata, so it does not need to start a Spark session just to ask whether data arrived recently.

19. What is `MERGE INTO`? Why is it needed for Gold but not Bronze?

    Answer: `MERGE` updates matching rows and inserts new ones in one atomic operation. Gold needs it because a window's row is revised as more data arrives, so the job must update the existing row instead of appending a duplicate. Bronze is append-only, because raw events never change once recorded, so a plain append is correct there.

20. What is idempotency? Name every layer where your pipeline enforces it.

    Answer: Idempotency means running an operation twice gives the same end result as running it once. My pipeline uses it in four places. First, Kafka keys by user id, which keeps ordering stable. Second, Silver deduplicates by event id and timestamp. Third, Gold uses MERGE on the window keys, so a replay updates rows instead of duplicating them. Fourth, the Postgres sync uses `ON CONFLICT`, so replays do not insert duplicates.

21. What is at-least-once vs. exactly-once delivery? Which do you claim, and why?

    Answer: At-least-once means no data is lost, but some messages may be processed twice. Exactly-once means each message is processed precisely once, which is hard to guarantee across Kafka, Spark, Delta, and Postgres. I claim at-least-once delivery with idempotent writes at each hop, which gives the same final result in practice. I do not claim formal exactly-once, because I have not proven it end to end.

22. What is a dead-letter queue or quarantine? Show where malformed events go in your pipeline.

    Answer: A dead-letter queue or quarantine holds records that fail validation, so they are not dropped silently and do not crash the job. In my Bronze stage, events that cannot be parsed, are missing required fields, or have an unknown event type are routed to a separate quarantine path. That keeps an audit trail of what was rejected and why.

23. What is an Airflow DAG? What does `catchup=False` do? Why `max_active_runs=1`?

    Answer: A DAG is a directed acyclic graph of tasks with dependencies. `catchup=False` stops Airflow from running every missed interval since the start date, which would fire a huge backlog of runs. `max_active_runs=1` ensures two runs never process the same window at the same time, which avoids overlapping writes.

24. Why does your Airflow use `DockerOperator`? What security risk comes with mounting the Docker socket?

    Answer: `DockerOperator` launches each Spark job in its own container, so tasks are isolated and resource use is visible. To do that, Airflow needs the Docker socket. The risk is that anyone with access to the socket can start a privileged container and effectively control the host. That is a real security trade-off. For local development it is acceptable, but I would not use it in production. In a cluster, I would use a Kubernetes-based operator instead.

25. What is dbt? Is it ETL or ELT? Explain `ref()` vs. `source()`.

Answer: dbt is a tool that runs SQL transformations in a warehouse, with dependency ordering, tests, and documentation. It is ELT, because it transforms data already loaded into the warehouse. `source()` points to a raw table that dbt does not create. `ref()` points to another dbt model, and dbt uses these references to build the dependency graph.

26. What is a star schema? Define grain. What is fan-out, and how do you detect it?

    Answer: A star schema has a central fact table with measurable events and foreign keys to dimension tables. Grain is what one row in the fact table represents. Fan-out happens when a join multiplies rows unexpectedly, so the fact count grows. I detect it by checking that the row count of `fact_user_sessions` equals the count of distinct session ids in the raw events, and by dbt's uniqueness test on the session key.

27. Why is `fact_user_sessions` one row per session? What would change at one row per event?

    Answer: The business questions in my dashboards are about sessions: conversion within a session, and cart abandonment. One row per session makes those answers direct. One row per event would be a different fact table, closer to raw events, and funnel questions would need more aggregation. Changing the grain changes every downstream query, so it should be decided first.

28. What dbt tests did you write? What does a `relationships` test check?

    Answer: I used `unique` and `not_null` on key columns such as the session id, and `relationships` tests to check that foreign keys exist in the dimension tables. A `relationships` test checks that every value in a column appears in the referenced model's column, which is the same as checking that a left join would not produce unmatched keys.

29. Why does `ON CONFLICT DO NOTHING` suit raw events but `DO UPDATE` suits Gold tables?

    Answer: A raw event never changes once recorded, so if the same event id appears again, skipping it is correct. Gold values, such as a window's count, can change as more data arrives, so the existing row should be replaced with the newer value. `DO NOTHING` is for immutable facts, and `DO UPDATE` is for values that get revised.

30. How would you handle schema evolution when a new field is added to events?

    Answer: I would make the new field optional with a default, so older events still parse. In Spark, I would add the field to the schema and handle nulls for old records. I would also update the producer and the schema file together. I should be honest that my schema is duplicated by hand in Python and Spark, with no automated check, so a schema registry with compatibility rules would be the proper fix.

31. How does your dashboard's freshness relate to the pipeline's schedule? What is the maximum lag?

    Answer: Each stage runs about every two minutes. The Gold data then syncs to Postgres in the same run, and Metabase queries Postgres live. So the lag is roughly the sum of the stage intervals and run durations, which I would estimate at several minutes at worst. I have not measured end-to-end lag precisely, so I would say that rather than quote a number.

32. How would you move this from local Docker to AWS? Map each component to a managed service.

    Answer: Kafka would become Amazon MSK. Spark would run on EMR or Databricks. Delta tables would sit on S3 in the same Delta format. Airflow would become MWAA. PostgreSQL would move to RDS or Aurora. Metabase could stay self-hosted or use a managed option. The main change is pointing the storage paths and connection settings at managed endpoints. The code and the Delta format stay the same.

PART E: Internship — Quantum Optimization (Qclairvoyance Quantum Labs)

1. What is a QUBO problem? Why is it a natural formulation for docking problems?

   Answer: QUBO means Quadratic Unconstrained Binary Optimization: we choose binary variables and minimise a quadratic function of them. Docking asks which placements of a molecule best fit a protein, and that can be encoded as binary choices with energy-like costs between them. The quadratic form captures pairwise interactions, which is what docking needs.

2. What is "encoding constraints as quadratic penalty functions"? Give a simple example.

   Answer: A hard constraint, such as "choose exactly one position," is added to the objective as a penalty that is large when the constraint is violated. For example, a penalty of P times (sum of x minus 1) squared is zero when exactly one variable is 1 and grows otherwise. Then a standard unconstrained solver can handle the problem, and a good solution satisfies the constraint.

3. What is molecular docking? What does binding energy mean, and why is lower better?

Answer: Molecular docking predicts how a small molecule binds to a target protein. Binding energy is the estimated free energy of that binding. Lower, more negative values mean a more stable binding, so a more favourable pose. My work used protein structure 1Y6R and reported a minimum binding energy of −8.57 across the solvers I benchmarked.

4. Compare exact solvers (SCIP, CPLEX) with heuristic solvers. When would you trust a heuristic?

   Answer: Exact solvers prove optimality but can take a very long time on large problems. Heuristics find good solutions quickly but offer no guarantee. I would trust a heuristic when the exact solver cannot finish, and when its solutions agree with exact results on smaller instances. In my work, I compared solution quality across solvers rather than assuming the exact one was always right.

5. What are Jaccard similarity and bitstring overlap? How did you use them?

   Answer: Jaccard similarity is the size of the intersection of two sets divided by the size of their union. For bitstrings, I treat the selected positions as sets and compare them. I used these metrics to compare solutions from exact and heuristic solvers and to see how similar their chosen configurations were.

6. Your result: 83.3% structural agreement between exact solvers. What does that imply about solution uniqueness?

   Answer: Agreement of 83.3% between exact solvers means their solutions were similar but not identical. That suggests several near-optimal configurations exist, so the optimum is not unique. In practice, a single objective value does not show which structure is physically meaningful.

7. Why is “objective value alone insufficient” for evaluating QUBO solutions?

   Answer: Two solutions can have nearly the same objective value but very different structures. If we only compare objective values, we might pick a solution that looks good on the cost function but is physically implausible. Structural comparison adds a second check on whether the solution makes sense for the domain.

8. What is quantum annealing vs. gate-based quantum computing? Which does QUBO target?

   Answer: Quantum annealing evolves a physical system toward the lowest-energy state, which maps naturally onto QUBO problems. Gate-based computing applies a sequence of quantum gates to a circuit. QUBO is a natural target for annealers, and it can also be mapped to gate-based approaches such as QAOA.

9. What is QAOA? In one sentence, what does it do?

   Answer: QAOA is a hybrid algorithm that alternates between a quantum circuit that prepares a state and a classical optimiser that tunes its parameters, to find low-cost solutions to combinatorial problems.

10. Your coursework includes Qiskit and PennyLane. Which one have you used in a real experiment, and what was the result?

    Answer: I have used these tools in coursework and in quantum-circuit exercises, including QAOA-style circuits. My internship work was on QUBO formulation and solver benchmarking, not on running circuits on hardware. I should be clear that my hands-on quantum circuit experience is at the coursework level, and I would be happy to walk through one of those exercises in detail.

11. What is a qubit? How does it differ from a classical bit?

    Answer: A classical bit is either 0 or 1. A qubit can be in a superposition of 0 and 1, and measuring it gives one outcome with probabilities determined by the state’s amplitudes.

12. What is superposition and entanglement? Give an intuitive example, not the math.

    Answer: Superposition is like a coin spinning in the air: it is not yet heads or tails until measured. Entanglement means two qubits share a joint state, so measuring one tells you something about the other, even though neither has a definite value alone before measurement.

13. Why is current quantum hardware noisy (NISQ)? What does that mean for practical use?

    Answer: Current devices have a limited number of qubits and high error rates, which is what NISQ, Noisy Intermediate-Scale Quantum, refers to. Errors build up as circuits get deeper, so practical use today is limited to small problems and hybrid methods that keep circuits short.

14. What did you learn from the internship that surprised you?

    Answer: [Personal reflection: fill in from actual experience.] A useful point from the work is that a good objective value is not enough on its own, and that comparing solvers structurally gave a more complete picture.

15. What part of the work did you do alone vs. with a team? How did you verify your results?

Answer: [Fill in: which parts were individual and which were team work.] For verification, I compared results across multiple solvers, measured agreement with Jaccard similarity and bitstring overlap, and checked solutions against the objective value.

PART F: Academic Background and Core CS Fundamentals

F1. Data Structures and Algorithms (expect this in every round)

1. What is the time complexity of binary search? Why does it require a sorted array?

   Answer: O(log n). Each comparison halves the remaining range. It needs sorted input because the decision to go left or right depends on the order of elements.

2. Compare arrays and linked lists: insertion, access, and memory layout.

   Answer: Arrays store elements contiguously, so access by index is O(1), but insertion in the middle is O(n) because elements must shift. Linked lists store nodes with pointers, so access is O(n), but insertion or deletion at a known node is O(1). Arrays are cache-friendly; linked lists are not.

3. How does a hash table work? What causes collisions, and how are they resolved?

   Answer: A hash function maps a key to an index in an array. Two keys can map to the same index, which is a collision. Common resolutions are chaining, where each slot holds a list, and open addressing, where the table probes for another free slot. With a good hash and a reasonable load factor, average lookups are O(1).

4. Explain a stack vs. a queue with one real-world use each.

   Answer: A stack is last-in, first-out, used for function call stacks and undo operations. A queue is first-in, first-out, used for task scheduling, such as the message queue in my URL shortener's click recording.

5. What is a binary search tree? What is the worst case, and how do balanced trees fix it?

   Answer: In a BST, left children are smaller and right children are larger. Search is O(log n) on average, but a sorted insertion sequence can make the tree a chain with O(n) operations. Self-balancing trees such as AVL or red-black trees keep the height logarithmic.

6. Explain BFS vs. DFS. When would you use each?

   Answer: BFS explores level by level using a queue, and is used for shortest paths in unweighted graphs. DFS goes deep first using a stack or recursion, and is used for cycle detection, topological sort, and connected components.

7. What is Dijkstra's algorithm? Why does it fail with negative edge weights?

   Answer: Dijkstra finds shortest paths from one source by repeatedly settling the closest unsettled node with a priority queue. It assumes that once a node is settled, its distance cannot be improved. A negative edge can break that assumption, so Bellman-Ford is used instead.

8. Explain dynamic programming with the 0/1 knapsack or longest common subsequence.

   Answer: Dynamic programming solves problems by combining solutions to overlapping subproblems and storing them. For 0/1 knapsack, let dp[i][w] be the best value using the first i items with capacity w. Then dp[i][w] = max(dp[i-1][w], value_i + dp[i-1][w - weight_i]) if the item fits. For LCS, the same idea compares prefixes of two strings.

9. What is a heap? Where would a priority queue be used in your projects?

   Answer: A heap is a tree where each parent is smaller (min-heap) or larger (max-heap) than its children, giving O(log n) insert and extract. A priority queue is built on it. Top-K problems and scheduling use it.

10. Solve on the whiteboard: two-sum, then reverse a linked list, then detect a cycle.

    Answer: Two-sum: use a hash map from value to index, and for each element check whether target minus it has been seen. O(n) time, O(n) space.

```
        def two_sum(nums, target):
            seen = {}
            for i, x in enumerate(nums):
                if target - x in seen:
                    return [seen[target - x], i]
                seen[x] = i
            return []
```

Reverse a linked list: iterate and reverse pointers, O(n) time, O(1) space. Cycle detection: Floyd's tortoise and hare, with a slow and fast pointer; if they meet, there is a cycle.

11. Solve: find the longest substring without repeating characters (sliding window).

    Answer: Keep a window [left, right] and a map of the last index of each character. When a repeated character is inside the window, move left past its previous index. Track the maximum window length. O(n) time.

    ```
        def length_of_longest_substring(s):
            last = {}
            left = best = 0
            for right, ch in enumerate(s):
                if ch in last and last[ch] >= left:
                    left = last[ch] + 1
                last[ch] = right
                best = max(best, right - left + 1)
            return best
    ```

12. Solve: merge k sorted lists (or explain the heap approach).

    Answer: Put the head of each list into a min-heap with its list index. Repeatedly pop the smallest, append it to the result, and push the next element from the same list. With N total elements and k lists, it is O(N log k).

13. What is the difference between a stable and an unstable sort? Give an example of each.

    Answer: A stable sort keeps the relative order of equal elements. Merge sort and Python's Timsort are stable. Quicksort in its usual form is unstable. Stability matters when you sort by one key after another.

14. What is Big-O? Compare O(n log n) and O(n^2) with an input size of 10^6.

    Answer: Big-O describes how cost grows with input size. For $n = 10^6$, n log n is about 2×10^7 operations, while n^2 is 10^{12}, so the quadratic algorithm would be around 50,000 times slower.

F2. Object-Oriented Programming

1. Explain the four pillars of OOP with a code example.

   Answer: Encapsulation hides internal state behind methods. Abstraction exposes only what matters. Inheritance lets a class reuse another class's behaviour. Polymorphism lets one interface have different implementations. In my producer code, `ClickstreamEvent` is a dataclass that encapsulates its fields, with a `to_json` method.

2. What is the difference between an abstract class and an interface?

   Answer: An abstract class can have implemented methods and state, and a class extends it. An interface defines a contract of methods, and in Python it is typically an abstract base class with only abstract methods. A class can implement several interfaces, but in most languages it extends only one class.

3. What is composition vs. inheritance? When would you prefer composition?

   Answer: Inheritance models an "is-a" relationship, and composition models "has-a", where one object holds another. I prefer composition when behaviour needs to be combined or swapped without a deep class hierarchy. For example, a service holding a repository is easier to test than a subclass of it.

4. What does `@dataclass(frozen=True)` do in Python, and why use it for config?

   Answer: It generates `__init__`, `__repr__`, and equality methods, and `frozen=True` makes instances immutable, so assigning to a field raises an error. Config should be read once and not changed later, so an immutable object prevents accidental changes.

5. What is a context manager (`with` statement)? Where have you used one?

   Answer: A context manager guarantees cleanup code runs when a block exits, even on an error. I use it for database cursors and file handling. In the Postgres sync code, the cursor is closed by the `with` block and the connection is closed in a `finally` block.

F3. Operating Systems

1. What is a process vs. a thread? When does Python's GIL matter?

   Answer: A process has its own memory space. Threads share memory within a process. Python's Global Interpreter Lock lets only one thread execute Python bytecode at a time, so CPU-bound threads do not run in parallel. It matters for CPU-heavy work, and it is less of a problem for I/O-bound work like network calls, which is why I use threads for

sending events and async I/O in the API.

2. What is a race condition? Show one example and how you prevent it.

   Answer: A race condition is when correctness depends on the timing of threads or processes. The classic example is two threads incrementing a shared counter, which can lose an update. I prevent it with a lock, such as `threading.Lock` around the counter, or with an atomic operation in the database.

3. What is a deadlock? What are the four Coffman conditions?

   Answer: A deadlock is when processes wait on each other forever. The four conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Breaking any one of them prevents deadlock, for example by acquiring locks in a fixed order.

4. What is virtual memory and paging?

   Answer: Virtual memory gives each process its own address space, mapped to physical memory by the OS. Paging divides memory into fixed-size pages. Pages not in use can be moved to disk, and a page fault loads them back when accessed.

5. How does a context switch work, and why is it expensive?

   Answer: The OS saves the registers and state of the running thread and loads another's. It is expensive because of the saved state, cache and TLB misses after the switch, and the scheduler work itself.

F4. DBMS and SQL

1. What is normalization? Explain 1NF, 2NF, and 3NF with an example.

   Answer: Normalization removes redundancy and update anomalies. 1NF requires atomic values with no repeating groups. 2NF removes partial dependencies on part of a composite key. 3NF removes transitive dependencies, so non-key columns depend only on the key. For example, storing both department_id and department_name in an employee table violates 3NF, because the name depends on the department, not the employee.

2. What is an ACID transaction? Give an example of each property.

   Answer: Atomicity: a transfer debits one account and credits another, or neither. Consistency: the total balance stays the same after a valid transfer. Isolation: two concurrent transfers do not see each other's partial state. Durability: after commit, the transfer survives a crash.

3. Explain INNER, LEFT, RIGHT, and FULL OUTER joins with a small example.

   Answer: If students and enrollments are joined on student_id, INNER returns only matching rows. LEFT returns all students, with nulls for those with no enrollments. RIGHT returns all enrollments. FULL returns all rows from both sides, with nulls where there is no match.

4. What is an index? When can an index slow things down?

   Answer: An index is a structure, usually a B-tree, that lets the database find rows by a column without scanning the table. Indexes slow writes, because every insert and update must also update the index, and they take storage.

5. Write a SQL query: second highest salary in an employee table. Then: top 3 products by revenue per category.

   Answer:

```sql
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

SELECT category, product_id, revenue
FROM (
  SELECT category, product_id, SUM(price) AS revenue,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY SUM(price) DESC) AS rn
  FROM sales
  GROUP BY category, product_id
) t
WHERE rn <= 3;
```

6. What is the difference between `WHERE` and `HAVING`?

   Answer: `WHERE` filters individual rows before grouping. `HAVING` filters groups after aggregation, so it can reference aggregate functions like `COUNT(*) > 5`.

7. What does `GROUP BY` require in the `SELECT` clause?

   Answer: Every selected column that is not an aggregate must appear in `GROUP BY`, because otherwise the database cannot tell which row's value to show for the group.

8. What is a window function? Write `ROW_NUMBER()` partitioned by user.

   Answer: A window function computes a value across related rows without collapsing them. Example:

```sql
SELECT user_id, event_timestamp,
       ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_timestamp) AS event_rank
FROM raw_events;
```

F5. Computer Networks and Web

1. What happens when you type a URL in a browser? Explain DNS, TCP, TLS, HTTP.

   Answer: The browser resolves the domain with DNS to an IP address. It opens a TCP connection, usually to port 443. For HTTPS, TLS negotiates encryption and checks the server's certificate. Then the browser sends an HTTP request, and the server responds with HTML, which the browser renders and fetches further resources for.

2. What is the difference between TCP and UDP? Give a use case for each.

   Answer: TCP is reliable and ordered, with connection setup and retransmission. UDP is connectionless and does not guarantee delivery. TCP suits web pages and APIs. UDP suits video calls and DNS queries, where a late packet is worse than a lost one.

3. What do HTTP status codes 200, 301, 307, 401, 404, 429, and 500 mean?

   Answer: 200 is success. 301 is a permanent redirect. 307 is a temporary redirect that keeps the method. 401 means unauthenticated. 404 means not found. 429 means too many requests. 500 means an unexpected server error.

4. What is CORS? Why does a browser enforce it?

   Answer: CORS is a mechanism where a server tells the browser which other origins may read its responses. The browser enforces the same-origin policy so that a malicious site cannot read data from another site using the visitor's session. My API uses an explicit allowlist of origins.

5. What is REST? What is the difference between `PUT` and `POST`? Is `GET` idempotent?

   Answer: REST models resources and uses HTTP methods to act on them. `PUT` replaces a resource at a known URL and is idempotent. `POST` creates a resource or triggers an action and is not idempotent. `GET` is idempotent, because repeating it does not change state.

6. What is a load balancer? Why do you need one with multiple API replicas?

   Answer: A load balancer distributes incoming requests across several backend instances. With multiple replicas, it spreads load and keeps the service up if one instance fails. It works well with my stateless API, since session state is kept in Redis or in the request.

F6. Machine Learning and Statistics (for AI/ML and Data Science candidates)

1. What is overfitting? How do you detect and reduce it?

   Answer: Overfitting is when a model learns noise in the training data and performs worse on new data. It shows up as a gap between training and validation performance. It can be reduced with more data, regularisation, dropout, early stopping, and simpler models. I used early stopping and dropout in the LSTM, and walk-forward validation to test on unseen windows.

2. Explain bias vs. variance.

   Answer: Bias is error from overly simple assumptions, so the model underfits. Variance is error from sensitivity to the training sample, so the model overfits. The goal is a model that balances the two.

3. What is cross-validation? Why use time-series split for financial data?

   Answer: Cross-validation rotates which part of the data is held out for testing. For time series, shuffling would leak future information into training, so I use time-ordered splits, like walk-forward windows, where the test period always comes after training.

4. Explain precision, recall, F1, and ROC-AUC. When would accuracy be misleading?

Answer: Precision is the share of predicted positives that are correct. Recall is the share of actual positives that are found. F1 is their harmonic mean. ROC-AUC measures ranking quality across thresholds. Accuracy is misleading when classes are imbalanced: a model that always predicts the majority class can look accurate.

5. What is gradient descent? What does the learning rate control?

   Answer: Gradient descent updates parameters in the direction that reduces the loss. The learning rate sets the step size. Too large, and training diverges. Too small, and it is very slow.

6. What is backpropagation at an intuitive level?

   Answer: Backpropagation uses the chain rule to pass the error signal backward through the layers, computing how much each weight contributed to the loss. Those gradients are then used by gradient descent.

7. What is regularization? Name L1 and L2 and their effects.

   Answer: Regularisation adds a penalty for large weights to discourage overfitting. L2 adds the sum of squared weights and shrinks all weights smoothly. L1 adds the sum of absolute weights and can drive some weights to exactly zero, which gives sparsity.

8. What is a p-value? What is a Type I vs. Type II error?

   Answer: A p-value is the probability of data at least this extreme if the null hypothesis is true. A Type I error is a false positive, rejecting a true null. A Type II error is a false negative, failing to reject a false null.

9. What is the Central Limit Theorem and why does it matter?

   Answer: The CLT says that the distribution of sample means approaches a normal distribution as the sample size grows, whatever the population distribution. It justifies using normal-based tests and confidence intervals on averages, as long as the sample is large enough.

10. What is the difference between supervised, unsupervised, and reinforcement learning? Give an example of each from your projects.

    Answer: Supervised learning learns from labelled examples, such as the LSTM predicting next-day returns from known outcomes. Unsupervised learning finds structure without labels, such as the clustering-style analysis I would use on user sessions. Reinforcement learning learns from rewards through interaction, which is the PPO and SAC portfolio agent.

F7. Data Engineering Basics (for Data Engineer candidates)

1. What is ETL vs. ELT? Which tool in your project is an ELT tool?

   Answer: ETL transforms data before loading it into the destination. ELT loads raw data first and transforms it in the warehouse. dbt is an ELT tool in my project, because it transforms data already loaded into Postgres.

2. What is data quality? Name five checks you would add to a clickstream pipeline.

   Answer: Data quality means data is accurate, complete, consistent, timely, and valid. Five checks I would add: required fields are not null, event_type is in the allowed set, timestamps are within a sane range, event ids are unique, and row counts from Bronze to Silver match after dedup. I already have schema validation and the dbt uniqueness and not-null tests. The timestamp range and count reconciliation checks are next steps.

3. What is a data warehouse vs. a data lake vs. a lakehouse?

   Answer: A warehouse stores structured, modelled data for SQL analytics. A data lake stores raw files cheaply in any format. A lakehouse combines both: files on cheap storage with table features like transactions and schema, as Delta Lake does in my project.

4. What is a slowly changing dimension? Which type would you use for a user’s address?

   Answer: A slowly changing dimension is a dimension whose attributes change over time. Type 1 overwrites the old value, Type 2 adds a new row with validity dates, and Type 3 keeps a previous column. For a user’s address where history matters for analysis, I would use Type 2. For a correction of a typo, Type 1 is enough.

F8. Academic Projects and Coursework Depth

1. Your GPA is 9.09. Which course did you find hardest, and what did you learn from it?

   Answer: [Fill in: the course you found hardest.] Use a real course and describe the concept you learned and how it helped in a project.

2. You list Quantum Computation, Deep Learning, and Operating Systems. Pick one and explain a concept you can teach in 3 minutes.

   Answer: I would pick Operating Systems and explain virtual memory: each process gets its own address space, pages map to physical frames, and a page fault loads a page from disk. It ties to why containers and Python processes are isolated.

3. Why did you choose Honors in Quantum Computing? What do you want to do with it?

   Answer: [Fill in: personal motivation.] I would connect it to my internship on QUBO optimisation, and say that I want to combine quantum-inspired optimisation with classical engineering, while being honest that I am still learning the hardware side.

4. Which of your projects would you rebuild differently today? What would you change?

   Answer: For the URL shortener, I would add idempotent click recording with unique event ids and a real readiness check. For LexAI, I would wire up the Google provider or remove it from the config, and use a real RAGAS evaluation with an LLM judge. For the portfolio engine, I would add survivorship-free data and a clean final holdout period.

5. What is the most complex bug you have ever debugged? Walk me through how you found it.

   Answer: The clearest example is the foreign-key bug in the URL shortener. The unit tests on SQLite passed because SQLite does not enforce foreign keys by default. I found it when the Postgres test suite rejected an insert. I traced the failing insert to a click recorded with a link id that did not exist, and fixed the logic so the id always came from a real link.

PART G: Coding Round Problems (expect 2 problems, 45–60 minutes)

Pick 2 of these depending on time:

1. Given an array of integers, return all pairs that sum to a target. Optimise to O(n).

   Answer: Use a hash set. For each number x, check whether target − x is in the set, and record pairs as you go. This returns pairs of values in O(n) time and O(n) space. For all index pairs, I would use the same hash map approach, storing indices.

   ```
   def pairs_with_sum(nums, target):
       seen, result = set(), []
       for x in nums:
           if target - x in seen:
               result.append((target - x, x))
           seen.add(x)
       return result
   ```

2. Implement an LRU cache with O(1) get and put. (Relevant to the URL shortener caching.)

   Answer: Use a hash map plus a doubly linked list. The map gives O(1) lookup of a node, and the list keeps recency order, with the most recent at the front. On get, move the node to the front. On put, insert at the front and, if over capacity, remove the tail.

   ```
   from collections import OrderedDict

   class LRUCache:
       def __init__(self, capacity):
           self.cap = capacity
           self.data = OrderedDict()

       def get(self, key):
           if key not in self.data:
               return -1
           self.data.move_to_end(key)
           return self.data[key]

       def put(self, key, value):
           if key in self.data:
               self.data.move_to_end(key)
           self.data[key] = value
           if len(self.data) > self.cap:
               self.data.popitem(last=False)
   ```

3. Implement a rate limiter: allow N requests per minute per IP. Explain fixed window vs. sliding window.

   Answer: A fixed window counts requests in each clock minute and resets at the boundary. Its weakness is that a client can send N requests at the end of one window and N at the start of the next. A sliding window keeps timestamps of recent requests and counts those within the last 60 seconds, which removes the burst problem at the cost of more memory. In production the counter lives in Redis so all replicas share it.

4. Given a stream of click events, count unique users per 5-minute window. (Relevant to clickstream.)

   Answer: Bucket each event by `timestamp // 300` to get its 5-minute window, and keep a set of user ids per bucket. The size of each set is the unique count. For a very large stream, I would use a probabilistic counter such as HyperLogLog, which trades a small error for constant memory.

```
from collections import defaultdict

def unique_users_per_window(events, width=300):
    buckets = defaultdict(set)
    for user_id, ts in events:
        buckets[ts // width].add(user_id)
    return {k * width: len(v) for k, v in buckets.items()}
```

5. Implement a function to encode a number to base62 and decode it back. (Relevant to URL shortener.)

   Answer: Repeatedly take the number modulo 62 and map the remainder to the alphabet, then reverse. Decoding multiplies and adds in the same base.

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

def encode(n):
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))

def decode(s):
    n = 0
    for ch in s:
        n = n * 62 + ALPHABET.index(ch)
    return n
```

6. Given stock prices for N days, find the maximum profit with at most one buy and one sell.

   Answer: Track the minimum price so far and the best profit if you sell today. O(n) time, O(1) space.

```
def max_profit(prices):
    best, low = 0, float("inf")
    for p in prices:
        low = min(low, p)
        best = max(best, p - low)
    return best
```

7. Given a string of brackets, validate balanced parentheses.

   Answer: Use a stack. Push opening brackets; on a closing bracket, check that the top matches and pop. The string is valid if the stack is empty at the end.

```
def is_valid(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

8. Implement top-K frequent elements using a heap.

Answer: Count frequencies with a dictionary. Then use a min-heap of size K, keeping the K largest counts. This is O(n log K).

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    return [x for x, _ in heapq.nlargest(k, counts.items(), key=lambda t: t[1])]
```

9. Given a list of intervals, merge overlapping intervals.

   Answer: Sort by start. Walk through the list, and either extend the last merged interval or append a new one.

```
def merge(intervals):
    intervals.sort()
    merged = []
    for s, e in intervals:
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

10. Given a grid, count islands (DFS or BFS).

    Answer: Scan the grid. When you find an unvisited land cell, start a DFS that marks all connected land as visited, and count one island. O(rows × cols).

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()
    def dfs(r, c):
        stack = [(r, c)]
        seen.add((r, c))
        while stack:
            x, y = stack.pop()
            for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nx, ny = x + dx, y + dy
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == "1" and (nx, ny) not in seen:
                    seen.add((nx, ny))
                    stack.append((nx, ny))
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
                dfs(r, c)
                count += 1
    return count
```

PART H: System Design and Architecture (light version for freshers)

1. Design a URL shortener for 1 million links per day. Start with the API, then the database, then caching.

   Answer: First I would clarify the read-to-write ratio and latency target. 1 million links a day is about 12 writes per second on average, with far more reads. API: `POST /links` to create and `GET /{slug}` to redirect. Database: a links table with a unique slug and an id from a sequence, plus a separate clicks table written asynchronously. Caching: Redis with cache-aside for slug lookups, negative caching for missing slugs, and a high TTL for stable links. Scaling: stateless API replicas behind a load balancer, read replicas for PostgreSQL, and rate limits on creation.

2. Design a rate limiter for a public API. Where would you store state?

   Answer: Each client has a counter per window. Store it in Redis, so all API replicas share one count. Use an atomic increment with an expiry for fixed windows, or a sorted set of timestamps for sliding windows. Return 429 with a Retry-After header when the limit is exceeded.

3. Design a basic clickstream pipeline that handles late events. Where does the watermark go?

Answer: Events go from producers to Kafka, keyed by user id. A streaming job reads them, parses and validates, and writes to a raw layer. Windowed aggregations use event time, not arrival time. The watermark goes on the event-time column, and it sets how long the job waits for late events before closing a window. Malformed events go to a quarantine path.

4. How would you design a recommendation feed for a shopping app with minimal latency?

   Answer: Precompute candidate recommendations offline, store them in a key-value store per user, and serve them from cache. At request time, filter out items that are out of stock or already viewed, and re-rank with a few fresh signals if needed. The budget is tight, so heavy models run offline.

5. How would you make an ML model in production handle drift?

   Answer: Monitor input distributions and live performance, compare them with the training baseline, and alert when they diverge. When drift is confirmed, retrain on recent data, evaluate the challenger against the champion on a holdout, and promote only if it is clearly better. My portfolio project does this with a rolling Sharpe alert and champion-challenger promotion, though I have not run it on live traffic long-term.

6. You have a slow SQL query on a table with 100 million rows. What do you check first?

   Answer: I would run `EXPLAIN ANALYZE` to see whether it does a full scan, which join or sort is slow, and whether row estimates are wrong. Then I would check for a missing index on the filter or join columns, whether the query returns more columns or rows than needed, and whether statistics are up to date. Partitioning by date is the next option if the table is time-based.

PART I: Behavioural and HR Round

1. Tell me about yourself in 90 seconds, ending with why you want this role.

   Answer: I am a final-year B.Tech student in Computer Science at SRM Institute of Science and Technology, with an honours track in Quantum Computing and a CGPA of 9.09. I have built four projects end to end: a scalable URL shortener with FastAPI, Redis, Celery and Docker; a legal question-answering system using hybrid retrieval and reranking; a reinforcement-learning portfolio engine with an MLOps pipeline; and a Spark and Kafka clickstream lakehouse with Airflow and dbt. I also interned on QUBO optimisation for molecular docking. I enjoy the parts where systems meet data and reliability, and I am looking for a role where I can keep building those systems at scale.

2. Why should we hire you over another final-year student with similar projects?

   Answer: I do not want to claim more than the projects show. What I can offer is that I have built across backend, data, and ML, and I know where each piece is weak. For example, I found and fixed a foreign-key bug that only appeared on Postgres, and I can tell you which of my results are not yet proven. I will be honest about gaps and I learn quickly.

3. Which role are you most interested in: Software Engineer, AI/ML Engineer, Data Engineer, or Data Scientist? Why? What would make you switch?

   Answer: My strongest project experience is in backend and data engineering, so I am most interested in Software Engineer or Data Engineer roles. I am also interested in AI/ML engineering, because of LexAI and the portfolio engine. I would move toward AI/ML if the team needed evaluation and model-serving work, and I would consider Data Scientist if the role were centred on experiment design and analysis.

4. Tell me about a time you disagreed with a teammate. How was it resolved?

   Answer: [Fill in a real example: the disagreement, what you did to understand the other view, and the outcome.] Use a real situation from a team project or college work, and describe the action you took.

5. Tell me about a project that failed or did not meet expectations. What did you learn?

   Answer: My clearest example is the foreign-key bug. My fast tests passed on SQLite, so the bug looked solved until the Postgres suite failed. I learned that tests have to run on the same database engine as production, and that passing tests are only as good as their environment. [Add any personal project setback you experienced.]

6. Tell me about a time you had to learn a technology quickly. How did you approach it?

   Answer: Several of my projects required tools I had not used before, such as Kafka, Spark, Airflow, and dbt for the clickstream lakehouse. I started with the official docs and a minimal working example, then built one stage at a time, checking each with a small test before adding the next. [Add a specific example with a timeline if you have one.]

7. How do you handle a bug that you cannot reproduce?

Answer: First I try to make it reproducible: I collect the exact input, environment, and logs, and check whether it depends on timing, data, or configuration. If it is a concurrency issue, I try to make the race happen deterministically, for example with a controlled number of concurrent requests. I write a failing test before fixing it, so the bug cannot quietly return.

8. Where do you see yourself in 3 years?

   Answer: I see myself as an engineer who owns a system end to end, from design to production monitoring, and who can explain the trade-offs clearly. I would like to go deeper in either backend or data platforms, and keep building in AI systems where evaluation is taken seriously.

9. Are you comfortable relocating? Working on-site? Working rotational shifts?

   Answer: [Fill in honest answer based on personal preference.]

10. Do you have any other offers or exams that may conflict with joining?

    Answer: [Fill in honest answer.] I would tell the panel any conflicts early so the timeline can be planned.

11. What questions do you have for us?

    Answer: I would ask: how the team measures the quality of what it ships, whether freshers get a mentor for the first few months, what a typical first project looks like, and which tools the team uses for monitoring and deployment.

PART J: Verification and Probing Points Specific to This Candidate's Resumes

Before the interview, reconcile these items so you do not get caught out, and so you can ask about them directly:

1. **Date conflicts:** Portfolio project is listed as Nov 2025–Feb 2026 in one resume and Jan–Mar 2026 in the other. LexAI is Jun–Nov 2025 in one and Jul–Sep 2025 in the other.

   Candidate's answer: [Confirm the correct dates from your own records and explain which resume version is current. Do not guess.]

2. **Contact and education details:** Two different email addresses and two education blocks (one with "Honors in Quantum Computing," one with "SRM IST-Kattankulathur").

   Candidate's answer: [Confirm the email you want recruiters to use and the exact education wording on your official transcript.]

3. **Metrics that need methodology:** Sharpe 1.4 vs. 0.6, $p < 0.05$, and the 83.3% structural agreement.

   Candidate's answer: The Sharpe numbers come from walk-forward test windows of the RL agent compared with benchmarks on the same windows. The p-value is from a two-sided t-test on monthly excess returns. The 83.3% is the structural agreement between exact solvers on the internship instances. I would be ready to explain each method, and I know my two project documents report different p-values, so I quote ranges and specific runs.

4. **Benchmark numbers:** The retrieval benchmark shows low absolute precision (about 0.02 at P@5).

   Candidate's answer: Absolute precision is low because the benchmark uses character-level spans and retrieves only five fixed-size chunks, so the relevant text is a small fraction of what is returned. The meaningful result is the relative improvement: recall@5 rose from about 0.003 for FAISS alone to about 0.19 with hybrid retrieval and reranking, on the same queries and chunker.

5. **Claims vs. implementation:** The resume says "RAG evaluation" and "RAGAS."

   Candidate's answer: The current evaluation uses lightweight token-overlap approximations of faithfulness and relevance, not the RAGAS library. I have a compatibility fix for RAGAS ready, but the active evaluation does not use LLM-judge metrics. I would describe it that way on the resume.

6. **"Horizontally scalable":**

   Candidate's answer: The API tier is stateless and can run as several replicas, with shared state in Redis. The database and Redis are still single instances, so they are the next bottleneck to address.

7. **Clickstream "exactly-once" claim:**

   Candidate's answer: The pipeline is at-least-once with idempotent writes at each hop. I do not claim formal exactly-once end to end.

Scoring Guide (quick reference for the panel)

Area	Strong	Average	Weak
Project depth	Explains trade-offs and bugs unprompted	Explains what, not why	Recites resume only
DSA / coding	Clean solution, complexity, edge cases	Solves with hints	Cannot start
CS fundamentals	Explains with examples	Definitions only	Confuses core terms
Role fit (SDE / AI-ML / DE / DS)	Clear evidence for at least one role	Evidence for two roles	No clear evidence
Communication	Structured, concise, asks clarifying questions	Gets there with prompts	Hard to follow
Honesty about limits	Names gaps and next steps	Partial	Overclaims

Suggested routing after interview: - Strong DSA + backend projects → Software Engineer track (URL Shortener, system design). - Strong ML + evaluation rigor → AI/ML Engineer track (LexAI, Portfolio Engine). - Strong pipeline and SQL depth → Data Engineer track (Clickstream Lakehouse). - Strong statistics and experiment design → Data Scientist track (Portfolio backtesting, benchmark analysis).

Prepared for internal panel use. Questions are based on typical Indian campus and off-campus fresher hiring patterns (aptitude + coding + technical + HR). Candidate answers are drafted from the project notes and resumes; items in [square brackets] need the candidate's own facts.